SMART INDIA HACKATHON · PROBLEM SIH26054

Student Viva Guide

Predictive Maintenance Analytics — every sidebar page, every button

Repo: predictive-maintenance-analytics · entry: `streamlit run app.py`
Generated 2026-09-06 from the live `app.py` control inventory.

Say this in viva, then stop: this app is a reliability add-on (sensor CSV → clean → map → Isolation Forest → RUL/risk → charts → insights → optional 3D/CAD twin). It is not a FADEC, not a GCS, not Forge, and not an OEE cockpit. SIH26054 is mapped honestly as health + digital twin + fault prediction + mission reliability on uploaded aero-piston / MALE-UAV sensor rows.

This PDF names every left-sidebar Pipeline radio page and, on each page, every button, download, file uploader, tab, expander, and main table — using the label shown in the UI, what it does, and the function/file that implements it.

0. How to use this booklet in viva

- Open the app. Point at the left sidebar Pipeline radio — there are exactly 13 pages.
- For any button the examiner clicks, this booklet has the exact label, the session keys it writes, and the Python function.
- If asked “did you build FADEC / GCS?” — answer no, then map SIH26054 to what you did build (below).
- If asked “19 checks or 20?” — 19. Named list is in chapter 2. Constant is `QUALITY_STAGE_COUNT = 19` in `src/quality_checks.py`.
- If asked pandas vs Polars — pandas is default industrial ETL; Polars is the opt-in columnar engine (no JVM). Quality report always runs in pandas after the engine step.
- If asked why the app crashed after “Suggest pack” — Streamlit forbids writing a widget-bound session key after that widget exists. We queue `_pending_*` keys instead.

0.1 What this is — and what it is not (SIH26054 / FADEC / GCS)

SIH26054 (DRDO) in this repo is the Aviation pack: digital twin, health, fault prediction, and mission reliability for aero piston engines on MALE UAVs. Code: pack id `aviation_uav_piston`, `sih="SIH26054"`, hero star in the sidebar Field picker, demo CSV `sample_data/aviation_uav_piston.csv`, 3D kind `aviation_uav`, hotspot “piston engine”. That is a reliability / twin mapping of the problem statement — not a flight-control product.

Not FADEC. Full Authority Digital Engine Control commands fuel, spark, and limits on a live engine. This app never writes a throttle, mixture, or ignition output. Isolation Forest scores uploaded (or simulated) rows; Random Forest predicts remaining useful life / risk. There is no closed-loop actuator path.

Not GCS. A Ground Control Station uplinks commands, shows a moving map, and flies the UAV. This app has no stick/rudder, no datalink, no flight plan, no lost-link procedure. Live Connect can stream JSON/MQTT/OPC-UA sensor payloads into the same PdM buffer — that is ingest, not command-and-control.

Also not: an airliner cabin twin; a jet / turbofan catalog; every airframe OEM; a certified airworthiness system; Forge (generic analytics OS); OEE Pulse (plant SaaS). Automotive is one generic ICE car — not every OEM, trim, or EV architecture. Oil is SIH26120 one SRP well — not a field SCADA historian.

Honest one-liner: “We implemented SIH26054 as sensor health + Isolation Forest + RUL + an offline UAV/piston twin and an optional Autodesk CAD twin. We did not implement FADEC or a GCS.”

0.2 Pipeline radio — all 13 pages

Created in `main()` as `st.sidebar.radio("Pipeline", list(pages.keys()), key="pipeline_page")`.

#	Sidebar label (exact)	page_* function	Role
1	**1. Upload & Clean**	`page_upload_clean`	Load CSV / URL, pandas|Polars clean, 19 checks
2	**2. Joins**	`page_data_integration`	Optional sensor ⋈ maintenance ⋈ cost
3	**3. Map sensors**	`page_map_sensors`	Canonical + pack extra column map
4	**4. Anomaly & RUL**	`page_ml_predictions`	Isolation Forest + Random Forest RUL
5	**5. Charts**	`page_explore_graphs`	Primary + extra Plotly views
6	**6. Insights**	`page_business_insights`	Inspect list, $ rates, Ask panel
7	**3D Twin**	`page_twin_3d`	Offline pack mesh, risk-colored hotspot
8	**CAD Twin (APS)**	`page_cad_twin`	Autodesk GuiViewer3D, URN, region tint
9	**Live Connect**	`page_live_connect`	Sim / poll / MQTT / OPC-UA stream
10	**7. Dashboard**	`page_dashboard_builder`	Power BI-style tiles + HTML export
11	**Email Report**	`page_email_report`	Preview / SMTP or demo-file send
12	**Ask**	`page_ai_assistant`	Same Ask path as Insights
13	**SQL lab**	`page_dwdm_sql`	Optional DWDM transforms + read-only SQL

Sidebar chrome (Industry pack + Gemini + status) is documented next, then each of the 13 pages.

1. Left sidebar chrome (always visible)

Rendered by `main()` before the selected page function. Pack selectbox and Pipeline radio are widget-bound. Gemini sits under the radio.

UI label (exact)	Kind	What it does	Function / file
Predictive Maintenance Analytics	Sidebar title	App name from **config.APP_TITLE**. Tagline under it states this is a reliability add-on, not a generic sales OS and not an OEE cockpit.	`main()` · `app.py` + `config.py`
Field	Selectbox	Industry pack picker (widget key `industry_pack`). Options: Plant / rotating machines (default); Aviation / aero piston / MALE UAV ★ SIH; Automotive powertrain; Oil well / sucker-rod pump. Switching pack changes extra sensor-map fields, KPIs, 3D mesh, and CAD hints. Do not write `st.session_state.industry_pack` after this widget exists.	`render_pack_sidebar()` · `app.py` → `get_pack()` · `src/industry_packs.py`
Suggest pack from columns	Button	Scores headers / machine_id prefixes via `suggest_pack()`, stores `pack_suggest`, queues `_pending_industry_pack`, then `st.rerun()`. Applied next run by `apply_pending_industry_pack()` before the selectbox is created.	`render_pack_sidebar()` · `app.py` → `suggest_pack()` · `src/industry_packs.py`
What this pack covers	Expander	Shows pack `scope` and `not_in_scope`. Aviation cites SIH26054. Automotive adds the OEM / trim / EV honesty note. Oil cites SIH26120.	`render_pack_sidebar()` · `app.py`
Pipeline	Radio	Left-sidebar page switch. Widget key `pipeline_page`. Thirteen labels, in order: 1. Upload & Clean; 2. Joins; 3. Map sensors; 4. Anomaly & RUL; 5. Charts; 6. Insights; 3D Twin; CAD Twin (APS); Live Connect; 7. Dashboard; Email Report; Ask; SQL lab. Queued jumps use `request_pipeline_page()` + `_pending_pipeline_page`.	`main()` · `app.py`
Paste Gemini API key	Password input	Widget key `gemini_key_input`. Does not persist until Save key. Env `GEMINI_API_KEY` still works.	`render_gemini_sidebar()` · `app.py` → `get_gemini_api_key()` · `src/insights_engine.py`
Save key	Button	Copies the paste box into session `gemini_api_key_override` via `persist_session_gemini_key()`. Empty paste warns. Session-only — not written to disk.	`render_gemini_sidebar()` · `app.py` → `persist_session_gemini_key()` · `src/insights_engine.py`
Test Gemini	Button	Calls Gemini with the resolved key/model (default `gemini-3.6-flash`; retired aliases remap). Success shows a short reply; failure is stored in `last_gemini_error` and shown — 404s are not swallowed.	`render_gemini_sidebar()` · `app.py` → `test_gemini_connection()` · `src/insights_engine.py`
Data: N rows / No data loaded	Status caption	After the radio: row count of `cleaned_df`, pack short name, mapped sensor count, RUL-label yes/no, table count, prediction count, saved-graph count.	`main()` · `app.py` → `mapping_status()` · `src/sensor_map.py`

1.1 Session keys — widget-bound vs safe writes

Streamlit raises `StreamlitAPIException` if you assign `st.session_state.<key>` after a widget with that key already exists in the same run. Only these functions may write widget-bound keys, because they run before those widgets: `init_session_state`, `apply_pending_industry_pack`, `apply_pending_pipeline_page`, `_seed_cad_urn_widgets`, `_seed_zip_root_widget` (frozenset `PRE_WIDGET_SESSION_FUNCS` in app.py). Smoke test `test_widget_bound_session_keys_are_not_written_after_widget` enforces this.

Widget-bound keys (do not assign after the widget exists)

Widget key	Widget	Rule
`industry_pack`	sidebar Field selectbox	Never assign after render_pack_sidebar(). Queue _pending_industry_pack.
`pipeline_page`	sidebar Pipeline radio	Never assign after main() radio. Queue _pending_pipeline_page.
`aps_urn_input`	CAD URN text input	Seed in _seed_cad_urn_widgets() before the widget.
`aps_zip_root_filename`	ZIP root text input	Seed in _seed_zip_root_widget() / PENDING_ZIP_ROOT.
`gemini_key_input`	Paste Gemini API key	Read on Save key; do not overwrite after widget.
`upload_sensor_files`	CSV uploader	Streamlit-owned.
`upload_url_input / upload_url_ingest_mode / upload_url_sql / upload_url_force_cache`	URL tab widgets	Streamlit-owned.
`upload_demo_pack / upload_maintenance_table`	Upload extras	Streamlit-owned.

Widget key	Widget	Rule
aps_cad_file_uploader / cad_gui_viewer / cad_region_pick / cad_region_include_related	CAD Twin widgets	Viewer is a custom component.
aps_asset_pick / twin_asset_pick / twin_source	Asset / risk pickers	Widget-bound.
dash_tile_kpis / dash_tile_charts / dash_tile_insights / dash_tile_twin3d / dash_tile_cad	Dashboard tiles	Widget-bound.
map_	Map sensors selectboxes	One key per canonical / extra field.

Safe / non-widget session keys (init_session_state defaults + page writes)

Session key(s)	Owner
raw_df / cleaned_df / uploaded_tables / data_loaded	Working tables
cleaning_summary / quality_report / insights	Clean outputs
sensor_mapping / predictions / anomaly_detector / rul_predictor / anomaly_summary	ML
optuna_rul / optuna_anomaly / rul_used_synthetic	Tuning + honesty flag
business_insights / asset_ranking / inspect_list / insight_index / polished_brief	Insights
chat_history / last_gemini_error / last_insight_error / gemini_api_key_override	Ask / Gemini
graph_folder / dashboard_tiles / pack_kpis / pack_suggest / cost_* / hours_if_stop	Charts + $
url_ingest_* / maintenance_table_attached / clean_engine	Ingest + engine
live_running / live_tick / live_buffer / live_source / live_cfg / live_asset_states / live_conn_id	Live Connect
aps_urn / aps_translate_log / aps_save_log / cad_selected_part / cad_region_report / cad_region_assign_log	CAD Twin session (not widget keys)
join_log / sql_result / email_preview / email_subject / email_recipient	Joins / SQL / Email
_pending_industry_pack / _pending_pipeline_page / _pending_aps_zip_root_filename	Pre-widget queues

Queue helpers: `request_industry_pack(id)` writes `_pending_industry_pack`; `request_pipeline_page(label)` writes `_pending_pipeline_page`. Both are applied at the top of `main()` before sidebar widgets.

2. Nineteen quality checks + pandas / Polars

Button label on Upload & Clean: Run industrial clean + 19 quality checks. Constant `QUALITY_STAGE_COUNT = 19`. The report table is built by `build_quality_report()` in `src/quality_checks.py`. Do not say “20 stages” or “GE only”.

2.1 Engine order (say this if asked pandas vs Polars)

- pandas (default): `clean_and_quality()` in src/data_cleaner.py runs industrial ETL (schema normalize, dedupe, sentinel-null fusion, casts, regression/median/mode impute, binning, sensor smoothing) then the 19 checks.
- Polars (opt-in, no JVM): `clean_with_polars()` in src/polars_clean.py does unique(), sentinel fusion, Float64 cast, median fill, IQR cap, then hands a pandas frame to the same `clean_and_quality()`. PySpark was removed.
- If Polars raises, the UI warns and falls back to pandas. Quality stages always run in pandas after the engine step.
- Session key `clean_engine` is "pandas" or "polars".

#	Check name (exact in the report table)	Pass / warn rule
1	**NULLS / Missing%**	FAIL if >20% missing, WARN if >5%.
2	**CONSTANT**	FAIL if any column has ≤1 distinct value.
3	**ZEROS**	WARN if >30% numeric zeros.
4	**DUPLICATES**	WARN if duplicate rows exist.
5	**Z-SCORE (>3σ)**	WARN on |z|>3 hits per numeric column.
6	**IQR OUTLIER**	WARN on Tukey 1.5·IQR outliers.
7	**ISOLATION FOREST**	Quality-time IF (contamination 0.08) — not the page-4 model.
8	**DBSCAN NOISE**	WARN on DBSCAN noise points.
9	**KMEANS DISTANCE**	WARN on points far from cluster centers.
10	**ROLLING IMPOSSIBLE JUMP**	FAIL on physically huge temp/vib/pressure/rpm steps.
11	**LAG / SENSOR CORRELATION**	WARN if temp/pressure/vib pairs are dead (~0 corr).
12	**GE EXPECTATIONS**	Nulls, temp [0,200], RUL trend, unique timestamp+asset, row count.
13	**YDATA CARDINALITY**	High-cardinality columns; optional ydata-profiling.
14	**CLEANLAB / DIRTY LABELS**	Outliers + failure=1/low-vib and failure=0/extreme-vib flags.
15	**PCA / CONCEPT DRIFT**	Early vs late explained-variance shift > 0.15.
16	**DOMAIN OPC / PHYSICS RULES**	Stuck sensor, leak suspect, unusual RUL, missed failures.
17	**ASSOCIATION RULE MINING**	HIGH-sensor co-occurrence baskets (Apriori-style proxy).
18	**SCHEMA / ROWCOUNT**	FAIL if 0 rows.
19	**TIMESTAMP PRESENT**	WARN if no timestamp/date/time/datetime column.

Sub-reports under the table: expanders Great Expectations — column expectations, ydata / Cleanlab / PCA drift / Association rules, and (when flags exist) Domain OPC / physics-rule violations. Those are the same 19-stage run, not extra stages.

3. Industry packs (sidebar Field) — honesty you must keep

One CSV → one pack. A machine_id prefix alone does not turn the mesh into an airplane. Use Suggest pack from columns or pick Field yourself.

Sidebar Field label	id / SIH	Demo CSV	Honest scope
Plant / rotating machines (default)	`plant_rotating`	`sensor_readings.csv`	Generic motor / pump / compressor. Not a full plant twin, not OEM motor CAD.
Aviation / aero piston / MALE UAV ★ SIH	`aviation_uav_piston · SIH26054`	`aviation_uav_piston.csv`	DRDO mapping: health + twin + fault prediction + mission reliability on one UAV + piston. Not FADEC, not GCS, not airliner, not turbofan, not every OEM.
Automotive powertrain	`automotive_powertrain`	`automotive_powertrain.csv`	One generic ICE car + gearbox. Not every OEM / trim / EV architecture.
Oil well / sucker-rod pump	`oil_srp · SIH26120`	`oil_srp.csv`	One beam-pump well (Oil India smart automation). Not every completion, not a reservoir model.

3.1 RUL honesty (shown on Map / Anomaly / Insights)

`config.RUL_HONESTY_CAPTION`: honest RUL needs a real `failure_within_days` (or alias). Sample CSV labels are simulated. Without a label the model trains on a sensor-degradation proxy — treat days-to-fail as a demo, not a plant or sortie forecast. Session flag `rul_used_synthetic`. Optuna tune RUL refuses to run without the label column.

3.2 CAD / twin honesty

Isolation Forest scores the asset, not each CAD solid. A High asset does not redden the whole engine. Tint applies only to name-matched nodes or to dbIds you assigned under Assign selected part to this region. Solid1 / mojibake names match nothing — the model stays default dark gray. Driver line: largest |z-score| among mapped sensors vs other rows of that column (`DRIVER_HONESTY` in src/cad_part.py). 3D Twin is a library mesh (no APS). CAD Twin needs APS secrets + a URN in your bucket.

4. Sidebar page: 1. Upload & Clean

Radio label 1. Upload & Clean · handler `page_upload_clean()` in `app.py`. Tables below list every button, download, uploader, tab, expander, and main table on this page.

Need a raw frame first (sample, upload, or URL). Clean writes cleaned_df used by later steps.

UI label (exact)	Kind	What it does	Function / file
File upload	Tab	Local multi-file ingest. Each file is registered in `uploaded_tables` and the last one becomes `raw_df`.	`page_upload_clean()` · `app.py`
From URL (cloud / DuckDB)	Tab	HTTPS / Google Drive / Kaggle ingest via DuckDB httpfs. Kaggle needs `KAGGLE_USERNAME` + `KAGGLE_KEY`.	`page_upload_clean()` → `_ingest_from_url_tab()` · `app.py`
Upload sensor / ops CSVs (multi-select OK)	File uploader	Accepts csv / tsv / xlsx / json. Key `upload_sensor_files`. Each file → `load_tabular_file()` → `register_table(stem)` → `raw_df`.	`page_upload_clean()` · `app.py` → `load_tabular_file()` · `src/data_integration.py`
Load Plant sample (default)	Button	Loads `sample_data/sensor_readings.csv`, registers sensors, plus maintenance/costs if present. Queues pack `plant_rotating`. Clears cleaned_df / predictions.	`page_upload_clean()` → `load_pack_sample(DEFAULT_PACK_ID)` · `app.py`
Or load a pack demo CSV	Selectbox	Key `upload_demo_pack`. Aviation / Automotive / Oil demo CSVs (Plant is the other button).	`page_upload_clean()` · `app.py` → `pack_label()` · `src/industry_packs.py`
Load pack demo	Button	Loads the selected pack CSV and queues that industry pack (SIH26054 aviation demo is here). Key `upload_load_pack_demo`.	`page_upload_clean()` → `load_pack_sample(demo_pack)` · `app.py`
Cloud data URL	Text input	Key `upload_url_input`. Direct HTTPS CSV/Parquet, Drive share, or Kaggle page / `kaggle://`.	`_ingest_from_url_tab()` · `app.py`
Ingest mode	Radio	Row limit (simple) | SQL slice (DuckDB). Key `upload_url_ingest_mode`. Stores `url_ingest_mode` as limit or sql.	`_ingest_from_url_tab()` · `app.py`
Row limit (0 = all rows — cap on free hosts for multi-GB files)	Number input	Shown in limit mode. Key `upload_url_row_limit`. 0 = no cap.	`_ingest_from_url_tab()` · `app.py`
SQL slice preset (plant / PdM templates)	Selectbox	Key `upload_url_sql_preset`. PdM presets: Last N rows; Last N days; Filter by machine_id; Filter by asset_id; Date range window; Random sample %; PdM — failures + near-failure window.	`_ingest_from_url_tab()` · `app.py` → `list_ingest_presets()` · `src/url_ingest.py`
Apply preset to SQL	Button	Writes the template (with {source} intact) into `url_ingest_sql`. Key `upload_apply_sql_preset`.	`_ingest_from_url_tab()` · `app.py` → `build_preset_sql()` · `src/url_ingest.py`
DuckDB SQL (use {source} for the resolved file path/URL)	Text area	Key `upload_url_sql`. Read-only SELECT/WITH only. Used when ingest mode is SQL slice.	`_ingest_from_url_tab()` · `app.py`
Always download to disk first (recommended for Google Drive / files > 100 MB)	Checkbox	Key `upload_url_force_cache`. Forced on when a SQL slice is used.	`_ingest_from_url_tab()` · `app.py`
Load from URL	Button	Primary. Resolves Drive/Kaggle, DuckDB-loads the slice, sets `raw_df`, registers a table, invalidates `cleaned_df`, stores `url_ingest_meta`. Key `upload_url_load`.	`_ingest_from_url_tab()` · `app.py` → `load_from_url()` · `src/url_ingest.py`
Attach maintenance table (optional — join in step 2)	Expander	Optional work-order / PM CSV so Joins can merge on machine_id.	`_render_maintenance_attach()` · `app.py`
Maintenance / work-order CSV	File uploader	Key `upload_maintenance_table`. Registers table name `maintenance`. Types csv / tsv / xlsx / xls.	`_render_maintenance_attach()` · `app.py` → `load_tabular_file()` · `src/data_integration.py`
Raw Data Preview	Table	Full `raw_df` in a 420 px scrollable grid (not a 10-row head). Caption shows row count.	`page_upload_clean()` → `_scrollable_full_table()` · `app.py`
Cleaning engine	Selectbox	pandas (default) and polars when installed. Stores `clean_engine`. Polars is columnar ETL, no JVM. PySpark was removed.	`page_upload_clean()` · `app.py` → `polars_available()` · `src/polars_clean.py`
Run industrial clean + 19 quality checks	Button	Primary. If Polars is selected, `clean_with_polars()` first (dedupe / sentinel-null / cast / median / IQR cap) then always `clean_and_quality()` (pandas industrial ETL + 19-stage report). Writes cleaned_df, cleaning_summary, quality_report, insights; registers table `cleaned`. Polars failure falls back to pandas with a warning.	`page_upload_clean()` · `app.py` → `clean_with_polars()` · `src/polars_clean.py` → `clean_and_quality()` · `src/data_cleaner.py` → `build_quality_report()` · `src/quality_checks.py`

UI label (exact)	Kind	What it does	Function / file
ETL / DWDM actions	Expander	Lists cleaner log lines (schema normalize, dedupe, imputation, binning, smoothing, plus any Polars actions).	`page_upload_clean()` · `app.py`
Cleaned Data Preview	Table	Full scrollable `cleaned_df` after a successful clean.	`page_upload_clean()` → `_scrollable_full_table()` · `app.py`
Download Cleaned CSV	Download	Exports `cleaned_df` as `cleaned_sensor_data_YYYYMMDD.csv`.	`page_upload_clean()` · `app.py`
19-Stage Quality Report	Table	DataFrame of the 19 named checks (see honesty chapter) with PASS / WARN / FAIL / INFO. Four metric tiles count those statuses.	`page_upload_clean()` · `app.py` · `QUALITY_STAGE_COUNT` in `src/quality_checks.py`
Great Expectations — column expectations	Expander	GE-style expectation rows (nulls, temp range, RUL trend, unique timestamp+asset, row count). Library optional; checks still run as a proxy.	`_render_quality_subreports()` · `app.py` · `_run_great_expectations()` · `src/quality_checks.py`
ydata / Cleanlab / PCA drift / Association rules	Expander	Four sub-blocks: ydata-profiling (optional), Cleanlab dirty-label / outlier flags, PCA early-vs-late variance drift, Apriori-style HIGH-sensor baskets.	`_render_quality_subreports()` · `app.py` · `src/quality_checks.py`
Domain OPC / physics-rule violations	Expander	Expanded when flags exist: stuck-sensor (temp>150 & vib<0.1), leak/sensor-fault, unusual RUL trend, missed-failure suspects.	`_render_quality_subreports()` · `app.py` · `_domain_opc_rules()` · `src/quality_checks.py`
Data Insights	Markdown block	Column-level stats from `analyze_columns()` after clean.	`page_upload_clean()` · `app.py` → `src/data_insights.py`

5. Sidebar page: 2. Joins

Radio label 2. Joins · handler `page_data_integration()` in `app.py`. Tables below list every button, download, uploader, tab, expander, and main table on this page.

Skip if you only have one CSV. Plant sample registers three tables so this page is demoable.

UI label (exact)	Kind	What it does	Function / file
Join mode	Radio	Two tables | Chain 3+ tables. Needs ≥2 registered tables (Plant sample loads sensors + maintenance + costs).	`page_data_integration()` · `app.py`
Left table	Selectbox	Key `join_left`. Two-table mode.	`page_data_integration()` · `app.py`
Right table	Selectbox	Key `join_right`. Excludes the left name.	`page_data_integration()` · `app.py`
Join type	Selectbox	inner — INNER JOIN — only matching keys; left — LEFT JOIN; right — RIGHT JOIN; outer — FULL OUTER JOIN. Labels come from JOIN_TYPES.	`page_data_integration()` · `app.py` · `JOIN_TYPES` · `src/data_integration.py`
Keys	Radio	Common columns | Different column names.	`page_data_integration()` · `app.py`
Join on	Multiselect	Shown when Keys = Common columns. Defaults to suggested intersection (machine_id preferred).	`page_data_integration()` · `app.py` → `suggest_join_keys()` · `src/data_integration.py`
Left key / Right key	Selectbox	Shown when Keys = Different column names.	`page_data_integration()` · `app.py`
Run join	Button	Primary. `join_two()` → writes cleaned_df and raw_df, registers table `joined`, stores join_log, shows meta JSON + first 20 rows.	`page_data_integration()` · `app.py` → `join_two()` · `src/data_integration.py`
Start (left)	Selectbox	Chain mode. Key `chain_left`.	`page_data_integration()` · `app.py`
Join #1 right	Selectbox	Key `chain_r1`.	`page_data_integration()` · `app.py`
Join #1 type	Selectbox	Key `how1`. Same JOIN_TYPES.	`page_data_integration()` · `app.py`
Join #1 keys	Multiselect	Key `k1`. Suggested common columns.	`page_data_integration()` · `app.py`
Join #2 right	Selectbox	Key `chain_r2`.	`page_data_integration()` · `app.py`
Join #2 type	Selectbox	Key `how2`.	`page_data_integration()` · `app.py`
Join #2 keys (must exist on interim + right)	Multiselect	Key `k2`.	`page_data_integration()` · `app.py`
Run join chain	Button	Primary. `join_many()` two steps; result becomes working table `joined`.	`page_data_integration()` · `app.py` → `join_many()` · `src/data_integration.py`
joined preview (head 20)	Table	Shown after either join button succeeds.	`page_data_integration()` · `app.py`
Last join log	Expander	JSON of the last join_log (keys, how, row counts).	`page_data_integration()` · `app.py`

6. Sidebar page: 3. Map sensors

Radio label 3. Map sensors · handler `page_map_sensors()` in `app.py`. Tables below list every button, download, uploader, tab, expander, and main table on this page.

Works on cleaned_df, else raw_df. Apply mapping overwrites cleaned_df.

UI label (exact)	Kind	What it does	Function / file
Sensors mapped / Timestamp / Asset id / RUL label	Metrics	From `mapping_status()`. Warns if no failure_within_days — RUL will use a degradation proxy.	`page_map_sensors() · app.py → mapping_status() · src/sensor_map.py`
Reading timestamp	Selectbox	Canonical field timestamp. Key `map_timestamp`. Suggests time / datetime aliases.	`page_map_sensors() · app.py · CANONICAL_FIELDS · src/sensor_map.py`
Machine / asset id	Selectbox	Canonical machine_id. Key `map_machine_id`. Aliases include uav_id / aircraft_id / well_id.	`page_map_sensors() · app.py`
Temperature / Vibration / Pressure / Rotational speed (RPM)	Selectboxes	Core PdM sensors. Keys `map_temperature`, `map_vibration`, `map_pressure`, `map_rpm`.	`page_map_sensors() · app.py · CANONICAL_FIELDS · src/sensor_map.py`
Days until failure (RUL label, optional)	Selectbox	Canonical failure_within_days. Key `map_failure_within_days`. Honest RUL needs this (or alias). Sample labels are simulated.	`page_map_sensors() · app.py · config.RUL_HONESTY_CAPTION`
Aviation extras (when pack = Aviation ★ SIH)	Selectboxes	Exhaust gas temp (EGT); Cylinder head temp (CHT); Engine oil pressure; Engine oil temperature; Airframe / engine hours; Altitude (ft); Throttle (%); Manifold pressure. Keys `map_<canonical>`.	`page_map_sensors() · app.py → extra_field_defs() · src/industry_packs.py`
Automotive extras (when pack = Automotive)	Selectboxes	Coolant temperature; Engine oil pressure; Engine load (%); Vehicle speed; Gear.	`page_map_sensors() · app.py → extra_field_defs()`
Oil / SRP extras (when pack = Oil / SRP)	Selectboxes	Polish-rod load; Tubing pressure; Casing pressure; Pump fillage (%); Strokes per minute; Production (bbl).	`page_map_sensors() · app.py → extra_field_defs()`
Apply mapping	Button	Primary. `apply_mapping()` renames/copies columns onto the working table. Saves `sensor_mapping`, writes cleaned_df, registers `cleaned`, shows first 8 mapped rows, reruns.	`page_map_sensors() · app.py → apply_mapping() · src/sensor_map.py`
mapped.head(8)	Table	Preview after Apply mapping (same run, before rerun).	`page_map_sensors() · app.py`

7. Sidebar page: 4. Anomaly & RUL

Radio label 4. Anomaly & RUL · handler `page_ml_predictions()` in `app.py`. Tables below list every button, download, uploader, tab, expander, and main table on this page.

Needs mapped sensors. Predictions drive Insights, 3D Twin, CAD tint, Dashboard, Email.

UI label (exact)	Kind	What it does	Function / file
Optuna trials (optional)	Slider	5–40, default 15. Used by the two Optuna buttons (anomaly uses min(12, trials)).	`page_ml_predictions()` · `app.py`
Detect anomalies + predict RUL	Button	Primary. Fits Isolation Forest (contamination from config or Optuna), annotates scores, fits Random Forest RUL, writes predictions per machine, attaches RUL columns, rebuilds the industrial brief (insights / ranking / inspect list). Warns if labels were synthetic.	`page_ml_predictions()` · `app.py` → `AnomalyDetector` · `src/ml/anomaly_detector.py` → `RULPredictor` · `src/ml/rul_predictor.py` → `build_industrial_brief()` · `src/insights_engine.py`
Optuna tune RUL	Button	Needs a real `failure_within_days` column — refuses synthetic-label Optuna. Stores `optuna_rul` JSON.	`page_ml_predictions()` · `app.py` → `tune_rul_model()` · `src/ml/optuna_tuner.py`
Optuna tune anomalies	Button	Tunes Isolation Forest contamination. Stores `optuna_anomaly`. You must click Detect anomalies + predict RUL again to apply.	`page_ml_predictions()` · `app.py` → `tune_anomaly_contamination()` · `src/ml/optuna_tuner.py`
Optuna RUL result / Optuna anomaly result	JSON	Shown after a successful tune.	`page_ml_predictions()` · `app.py`
Failure Predictions	Markdown list	One risk-colored line per asset (High / Medium / Low) from `predictions`.	`page_ml_predictions()` · `app.py`
MAE (days) / RMSE (days) / R² Score	Metrics	RUL hold-out metrics when a real label existed.	`page_ml_predictions()` · `app.py` · `RULPredictor.metrics`
Top 10 Features for RUL Prediction	Chart	Horizontal Plotly bar of Random Forest importances.	`page_ml_predictions()` · `app.py` → `style_bar_figure()` · `src/graphs/layout.py`
Anomaly Detection Summary	Metrics	Records Analyzed, Anomalies Found, Anomaly Rate % from Isolation Forest summary.	`page_ml_predictions()` · `app.py` · `AnomalyDetector.summary()`

8. Sidebar page: 5. Charts

Radio label 5. Charts · handler `page_explore_graphs()` in `app.py`. Tables below list every button, download, uploader, tab, expander, and main table on this page.

Generate buttons save into graph_folder for Dashboard export.

UI label (exact)	Kind	What it does	Function / file
X-Axis	Selectbox	Prefers timestamp when present.	`page_explore_graphs()` · `app.py` → `get_axis_options()`
Sensor / metric	Selectbox	Numeric column. Defaults to the pack's preferred metric (EGT on Aviation, vibration/temp on Plant).	`page_explore_graphs()` · `app.py` → `preferred_y_metric()` · `src/industry_packs.py`
Filter by Machine	Multiselect	Defaults to all machine_id values.	`page_explore_graphs()` · `app.py`
Pack KPI tiles + health / KPI bar charts	Metrics + charts	Aviation tiles include engine health, mission reliability, remaining mission hours, EGT margin (SIH26054). Plant / Auto / Oil have their own KPI ids.	`page_explore_graphs()` · `app.py` → `compute_pack_kpis()` · `src/pack_kpis.py` → `create_asset_health_chart` / `create_pack_kpi_bars` · `src/graphs/pack_kpis.py`
Generate Sensor over time	Button	Primary chart. Key `gen_time_series`. Plotly time series; saved into graph_folder.	`page_explore_graphs()` · `app.py` → `create_time_series_chart()` · `src/graphs/time_series.py` → `save_graph_to_folder()` · `src/dashboard_builder.py`
Generate Anomaly flags	Button	Key `gen_anomaly_flags`. Isolation Forest flags on the timeline (needs a trained detector).	`page_explore_graphs()` · `app.py` → `create_anomaly_flags_chart()` · `src/graphs/anomaly_flags.py`
Generate Risk by asset	Button	Key `gen_risk_by_asset`. Predicted RUL days colored High / Medium / Low (needs predictions).	`page_explore_graphs()` · `app.py` → `create_risk_by_asset_chart()` · `src/graphs/risk_by_asset.py`
More sensor views (optional)	Expander	Secondary Plotly views. Each button is icon + name from GRAPH_TYPES.	`page_explore_graphs()` · `app.py` · `GRAPH_TYPES` · `src/graphs/__init__.py`
Sensor Correlation Heatmap	Button	Key `gen_heatmap`.	`page_explore_graphs()` · `app.py` → `create_correlation_heatmap()` · `src/graphs/heatmap.py`
⚠ **Anomaly Scatter Plot**	Button	UI icon is the warning emoji + name. Key `gen_anomaly_scatter`.	`page_explore_graphs()` · `app.py` → `create_anomaly_scatter()` · `src/graphs/anomaly_scatter.py`
Distribution Histogram	Button	Key `gen_histogram`.	`page_explore_graphs()` · `app.py` → `create_distribution_histogram()` · `src/graphs/histogram.py`
Box Plot by Machine	Button	Key `gen_boxplot`.	`page_explore_graphs()` · `app.py` → `create_boxplot_by_machine()` · `src/graphs/boxplot.py`
Rolling Average Trend	Button	Key `gen_rolling_trend`.	`page_explore_graphs()` · `app.py` → `create_rolling_trend()` · `src/graphs/rolling_trend.py`
{title} — {created_at}	Expander	One expander per saved graph in graph_folder, showing the Plotly figure.	`page_explore_graphs()` · `app.py` → `get_graph_folder()` · `src/dashboard_builder.py`
Remove {g_id}	Button	Deletes that entry from `st.session_state.graph_folder` and reruns. Key `rm_{g_id}`.	`page_explore_graphs()` · `app.py`

9. Sidebar page: 6. Insights

Radio label 6. Insights · handler `page_business_insights()` in `app.py`. Tables below list every button, download, uploader, tab, expander, and main table on this page.

Refresh brief is enough after ML. Ask panel is the same function as the Ask page.

UI label (exact)	Kind	What it does	Function / file
$ / hour downtime (label follows pack)	Number input	Plant: $ / hour downtime. Aviation: $ / hour of lost sortie time. Automotive: $ / hour vehicle down. Oil: $ / hour deferred production. Session key `cost_per_hour`. Dollars are only from rates you type.	`page_business_insights()` · `app.py` · pack `cost_hour_label` · `src/industry_packs.py`
$ / unit lost production (label follows pack)	Number input	Aviation: $ / aborted mission. Session `cost_per_unit`.	`page_business_insights()` · `app.py`
Hours if a high-risk asset stops	Number input	Default 8 h is an assumed stop — not a booked outage. Session `hours_if_stop`.	`page_business_insights()` · `app.py`
Refresh brief	Button	Primary. Rebuilds inspect list / ranking / pack KPI cards via `_refresh_brief()` and rebuilds the LlamaIndex / TF-IDF retrieval index.	`page_business_insights()` · `app.py` → `build_industrial_brief()` · `src/insights_engine.py` → `make_insight_index()`
Gemini polish (wording only)	Button	Rewords the rule-based brief. Numbers stay from the table. Needs a key; otherwise shows the no-key message.	`page_business_insights()` · `app.py` → `polish_brief_with_gemini()` · `src/insights_engine.py`
Asset rank	Table	Columns: rank, machine_id, risk_level, predicted_rul_days, mean_anomaly_score, anomaly_count, anomaly_rate_pct, n_rows.	`page_business_insights()` · `app.py`
Gemini brief	Markdown	Shown after a successful polish. Stored in `polished_brief`.	`page_business_insights()` · `app.py`
Insight cards (title — severity)	Markdown	Inspect-this-week / slow-running / $ at risk / pack KPI cards. Grounded in this upload, not generic advice.	`page_business_insights()` · `app.py` → `generate_business_insights()` / `insight_cards_from_kpis()`
Ask this upload (chat input)	Chat input	Placeholder: Ask about an asset, inspect list, or $ at risk on this upload.... Answers use this table + Isolation Forest / RUL columns — not a general LLM essay.	`render_ask_panel()` · `app.py` → `ask_with_index()` · `src/insights_engine.py`
Clear Ask	Button	Clears `chat_history`. Shown only when history is non-empty.	`render_ask_panel()` · `app.py`

10. Sidebar page: 3D Twin

Radio label 3D Twin · handler `page_twin_3d()` in `app.py`. Tables below list every button, download, uploader, tab, expander, and main table on this page.

No buttons — radio / selectbox / mesh only. Risk is read-only from ML or Live.

UI label (exact)	Kind	What it does	Function / file
Risk source	Radio	Anomaly & RUL (batch) and/or Live Connect (streaming). Key `twin_source`. Defaults to Live when streaming is on.	`page_twin_3d() · app.py → asset_states_from_predictions() · src/twin3d.py`
Asset	Selectbox	Key `twin_asset_pick`. Drives which hotspot is colored.	`page_twin_3d() · app.py`
Risk	Metric	High / Medium / Low of the selected asset.	`page_twin_3d() · app.py`
Preview risk	Selectbox	High | Medium | Low. Shown only when there are no predictions yet (demo mesh).	`page_twin_3d() · app.py`
3D mesh (three.js, no CDN)	Embedded HTML	Pack mesh: Plant motor (drive-end bearing hotspot); Aviation UAV + piston (SIH26054); generic ICE car; SRP beam pump. High = red blink, Medium = amber, Low = green. Offline — vendored three.js.	`page_twin_3d() → _render_twin() · app.py → build_twin_html() · src/twin3d.py`

11. Sidebar page: CAD Twin (APS)

Radio label CAD Twin (APS) · handler `page_cad_twin()` in `app.py`. Tables below list every button, download, uploader, tab, expander, and main table on this page.

Most buttons live on this page. Public viewer.autodesk.com URNs will not load.

UI label (exact)	Kind	What it does	Function / file
Asset	Selectbox	Key `aps_asset_pick`. Strip above the viewer. Risk / RUL come from Anomaly & RUL or Live Connect.	`render_cad_part_card()` · `app.py`
Load CAD model	Button	Primary. Key `aps_load_cad_btn`. Disabled until APS credentials + a translated (non-public-viewer) URN exist. Fetches a 2-legged token and opens GuiViewer3D. Also persists the URN for this pack.	`render_cad_part_card()` · `app.py` → `get_access_token()` / `_render_cad_viewer()` · `src/aps_viewer.py`
Open Anomaly & RUL	Button	Key `cad_open_anomaly_rul`. Queues `pipeline_page = 4`. Anomaly & RUL via `request_pipeline_page()` and reruns. Does not write the radio key after the widget exists.	`render_cad_part_card()` · `app.py` → `request_pipeline_page()` · `app.py`
Region for the clicked part	Selectbox	Heads/exhaust | Oil | Rotating/crank | Intake | Other. Key `cad_region_pick`. Defaults from the sensor driver region.	`render_cad_region_assign()` · `app.py` · `REGIONS` · `src/cad_regions.py`
Assign selected part to this region	Button	Primary. Key `cad_assign_region_btn`. Disabled until a part is clicked (dbId) and a URN exists. Saves {pack + URN → region → dbIds} to `data/cad_region_map.json`. That map is what turns Solid1 red — Isolation Forest does not score CAD parts.	`render_cad_region_assign()` · `app.py` → `assign_region()` · `src/cad_regions.py`
Clear region map	Button	Key `cad_clear_region_btn`. Forgets every assigned part for this model. Disabled when the map is empty.	`render_cad_region_assign()` · `app.py` → `clear_region_map()` · `src/cad_regions.py`
Also tint related regions	Checkbox	Key `cad_region_include_related`. e.g. a heads/exhaust driver also tints assigned oil parts.	`render_cad_region_assign()` · `app.py` → `assigned_theme_plan()` · `src/cad_regions.py`
Mapped sensors — latest row for this asset	Metrics grid	Four metrics per row from the latest mapped sensor values. Caption marks the clicked-part hint.	`render_cad_sensor_grid()` · `app.py` → `latest_asset_sensors()` · `src/cad_part.py`
CAD source — upload, translate, URN, save	Expander	Admin panel below the viewer so it never pushes GuiViewer3D off-screen. Expanded when no URN yet.	`page_cad_twin()` → `_render_cad_source_panel()` · `app.py`
CAD file for Model Derivative	File uploader	Key `aps_cad_file_uploader`. Streamlit max 300 MB. STEP / ZIP / Revit / Fusion / IFC / Inventor / SolidWorks / DWG / OBJ / STL …	`_render_cad_source_panel()` · `app.py` · `CAD_UPLOAD_EXTENSIONS` · `src/aps_viewer.py`
ZIP root filename (auto-filled for zipped STEP — leave as-is)	Text input	Key `aps_zip_root_filename`. Prefill happens in `_seed_zip_root_widget()` before this widget exists. Do not assign the key after.	`_render_cad_source_panel()` · `app.py` → `resolve_zip_root_filename()` · `src/aps_viewer.py`
Translate to SVF / get URN	Button	Primary. Key `aps_translate_btn`. Uploads bytes to your OSS bucket and runs Model Derivative. Writes `aps_urn` / `aps_urn_input`, saves the pack URN, forces viewer load. viewer.autodesk.com is Autodesk's public site — it does not put the file in your bucket.	`_render_cad_source_panel()` · `app.py` → `translate_cad_bytes()` · `src/aps_viewer.py`
Translated model URN (base64) or viewer.autodesk.com URL	Text input	Key `aps_urn_input`. Seeded by `_seed_cad_urn_widgets()` from APS_MODEL_URN, saved pack JSON, or session. Public-viewer URNs are extracted but blocked from loading with a 2-legged token.	`_render_cad_source_panel()` · `app.py` → `extract_model_urn()` · `src/aps_viewer.py`
Save URN for this pack	Button	Key `aps_save_urn_btn`. Writes `data/cad_urns.json` for the current industry pack. Public-viewer URNs are refused. Render redeploys wipe disk — set APS_MODEL_URN too.	`_render_cad_source_panel()` · `app.py` → `save_urn_for_pack()` · `src/aps_viewer.py`
Delete saved URN	Button	Key `aps_delete_urn_btn`. Clears the pack entry in cad_urns.json on this server.	`_render_cad_source_panel()` · `app.py` → `delete_urn_for_pack()` · `src/aps_viewer.py`
How to generate a URN (APS app + this page)	Expander	Six-step APS honesty: create an APS app, set Render secrets, translate on this page into your bucket, do not trust viewer.autodesk.com, required OAuth scopes, Save/Delete vs APS_MODEL_URN after redeploy. Also documents GuiViewer3D toolbar and Solid1 name sanitizing.	`page_cad_twin()` · `app.py`

12. Sidebar page: Live Connect

Radio label Live Connect · handler `page_live_connect()` in `app.py`. Tables below list every button, download, uploader, tab, expander, and main table on this page.

Start live feeds live_asset_states into 3D Twin.

UI label (exact)	Kind	What it does	Function / file
Source	Radio	Simulator | Poll CSV URL | MQTT (live) (if paho-mqtt) | OPC-UA (live) (if asyncua). Stores `live_source` as sim / poll / mqtt / opcua.	`page_live_connect()` · `app.py` · `mqtt_available / opcua_available` · `src/live_sources.py`
Machines	Slider	Simulator only. 2–8 assets.	`page_live_connect()` · `app.py`
Failing asset (drifts to failure)	Selectbox	Simulator only. That machine's sensors ramp toward failure.	`page_live_connect()` · `app.py` → `simulate_batch()` · `src/live_connect.py`
Ramp ticks to failure	Slider	Simulator only. 10–120 ticks.	`page_live_connect()` · `app.py`
CSV feed URL	Text input	Poll mode. Default http://localhost:8000/sensor_readings.csv.	`page_live_connect()` · `app.py` → `poll_url_increment()` · `src/live_connect.py`
Rows per poll	Slider	Poll mode. 5–200.	`page_live_connect()` · `app.py`
Broker host / Port / Topic (wildcards OK)	Inputs	MQTT mode. Default topic pdm/sensors/#. Expects JSON with machine_id + sensors.	`page_live_connect()` · `app.py` → `start_mqtt()` · `src/live_sources.py`
Username (optional) / Password (optional)	Text inputs	MQTT auth. Password is type=password.	`page_live_connect()` · `app.py`
OPC-UA endpoint	Text input	Default opc.tcp://127.0.0.1:4840/freeopcua/server/.	`page_live_connect()` · `app.py` → `start_opcua()` · `src/live_sources.py`
Node map — sensor=node_id, comma-separated	Text area	Example temperature=ns=2;i=2, vibration=ns=2;i=3.	`page_live_connect()` · `app.py` → `parse_node_map()` · `src/live_sources.py`
Machine id label	Text input	OPC-UA asset name (default opcua-asset).	`page_live_connect()` · `app.py`
▶ Start live	Button	Primary. Stops any prior source, starts sim / poll / MQTT / OPC-UA, clears buffer, sets live_running True. MQTT/OPC connection id stored in `live_conn_id`.	`page_live_connect()` · `app.py` → `start_mqtt / start_opcua` · `src/live_sources.py`
□ Stop	Button	Sets live_running False and `stop_source()`.	`page_live_connect()` · `app.py` → `stop_source()` · `src/live_sources.py`
Reset buffer	Button	Clears live_buffer, live_tick, live_asset_states.	`page_live_connect()` · `app.py`
Live chart	Selectbox	temperature | vibration | pressure | rpm. Session `live_sensor_view`.	`page_live_connect()` · `app.py`
Live asset status	Table	Per-machine risk from the rolling buffer. Feeds 3D Twin and Insights. Metrics: Rows buffered, Machines, Worst asset.	`_live_body()` · `app.py` → `compute_live_status()` · `src/live_connect.py`
Live {sensor} (last N readings)	Chart	Plotly line of the selected sensor, last 300 rows, colored by machine_id. Auto-refresh ~2 s via st.fragment when running.	`_live_body()` · `app.py`

13. Sidebar page: 7. Dashboard

Radio label 7. Dashboard · handler `page_dashboard_builder()` in `app.py`. Tables below list every button, download, uploader, tab, expander, and main table on this page.

Checkboxes toggle tiles; the only download is the HTML export.

UI label (exact)	Kind	What it does	Function / file
KPI strip	Checkbox	Tile id kpis. Key `dash_tile_kpis`. Pack health / mission reliability / fillage.	`page_dashboard_builder()` · `app.py` · `TILE_SPECS` · `src/dashboard_composer.py`
Charts	Checkbox	Tile id charts. Key `dash_tile_charts`. Saved Plotly views + pack health bars.	`page_dashboard_builder()` · `app.py`
Insights	Checkbox	Tile id insights. Key `dash_tile_insights`. Inspect-this-week and pack cards.	`page_dashboard_builder()` · `app.py`
3D twin	Checkbox	Tile id twin3d. Key `dash_tile_twin3d`. Offline pack mesh.	`page_dashboard_builder()` · `app.py` → `board_twin_html()` · `src/dashboard_composer.py`
CAD twin (APS)	Checkbox	Tile id cad. Key `dash_tile_cad`. Real CAD when Render secrets + URN exist; otherwise a placeholder.	`page_dashboard_builder()` · `app.py` → `board_cad_html()` / `cad_placeholder_html()` · `src/dashboard_composer.py`
KPI strip (section)	Metrics	Rendered when the KPI checkbox is on.	`page_dashboard_builder()` · `app.py` → `compute_pack_kpis()` · `src/pack_kpis.py`
Charts (section)	Charts	Pack figures plus any graphs saved on page 5.	`page_dashboard_builder()` · `app.py` → `board_pack_charts()`
Insights (section)	Cards	Up to 8 insight cards. Empty until Anomaly & RUL / Insights have run.	`page_dashboard_builder()` · `app.py`
3D twin (section)	Embedded HTML	Same pack mesh as the 3D Twin page.	`page_dashboard_builder()` · `app.py`
CAD twin (APS) (section)	Embedded HTML	GuiViewer3D when ready; otherwise the waiting-for-credentials / URN placeholder.	`page_dashboard_builder()` · `app.py` → `cad_slot_status()`
Export dashboard (HTML) — KPIs + charts + 3D (CAD slot placeholder)	Download	Label becomes "… + CAD" when APS is ready. Writes a standalone HTML via `compose_dashboard_html()`. File `reliability_dashboard_YYYYMMDD.html`.	`page_dashboard_builder()` · `app.py` → `compose_dashboard_html()` · `src/dashboard_composer.py`

14. Sidebar page: Email Report

Radio label Email Report · handler `page_email_report()` in `app.py`. Tables below list every button, download, uploader, tab, expander, and main table on this page.

Send Email is demo-file unless SMTP is configured.

UI label (exact)	Kind	What it does	Function / file
Recipient Email	Text input	Placeholder manager@company.com.	`page_email_report()` · `app.py`
Manager Name	Text input	Default Manager.	`page_email_report()` · `app.py`
Email Subject	Text input	Default Predictive Maintenance Report — YYYY-MM-DD.	`page_email_report()` · `app.py`
Additional Notes (optional)	Text area	Appended to the generated body. First 5 insight cards are also appended when present.	`page_email_report()` · `app.py`
Generate Email Preview	Button	Primary. Builds a text body from predictions, anomaly summary, insights, pack KPIs. Stores email_preview / email_subject / email_recipient.	`page_email_report()` · `app.py` → `generate_email_body()` · `src/email_report.py`
Email Preview	Text block	Shown after Generate. This is what will be sent or downloaded.	`page_email_report()` · `app.py`
Send Email	Button	Primary. SMTP if SMTP_USER is set and EMAIL_DEMO_MODE is false; otherwise demo mode writes a .txt under output/emails/ and says so. SMTP failure also falls back to file. Do not claim “we emailed the plant” unless SMTP is configured.	`page_email_report()` · `app.py` → `send_email()` · `src/email_report.py`
Download Email as TXT	Download	File maintenance_report_YYYYMMDD.txt.	`page_email_report()` · `app.py`

15. Sidebar page: Ask

Radio label Ask · handler `page_ai_assistant()` in `app.py`. Tables below list every button, download, uploader, tab, expander, and main table on this page.

No unique buttons beyond Clear Ask — same render_ask_panel as Insights.

UI label (exact)	Kind	What it does	Function / file
Ask this upload (chat input)	Chat input	Same path as Insights. Placeholder: Ask about an asset, inspect list, or $ at risk on this upload…. Grounded in this upload only.	`page_ai_assistant()` → `render_ask_panel()` · `app.py` → `ask_with_index()` · `src/insights_engine.py`
Clear Ask	Button	Clears chat_history. Hidden until there is history.	`render_ask_panel()` · `app.py`

16. Sidebar page: SQL lab

Radio label SQL lab · handler `page_dwdm_sql()` in `app.py`. Tables below list every button, download, uploader, tab, expander, and main table on this page.

Optional warehouse lab. Core PdM does not require this step.

UI label (exact)	Kind	What it does	Function / file
DWDM concept map	Table	Ten rows from DWDM_CONCEPTS: ETL, Joins, 19-stage quality, binning, smoothing, association rules, outlier mining, concept drift, OLAP aggregation, SQL.	`page_dwdm_sql()` · `app.py` · `DWDM_CONCEPTS` · `src/dwdm_sql.py`
Bin columns	Multiselect	Creates `{col}_dwdm_bin` via qcut.	`page_dwdm_sql()` · `app.py` → `apply_dwdm_transforms()` · `src/dwdm_sql.py`
Smooth columns	Multiselect	Rolling-mean `{col}_dwdm_smooth`. Defaults to temp/vib-like names.	`page_dwdm_sql()` · `app.py`
Z-normalize	Multiselect	Adds z-scored copies of selected numeric columns.	`page_dwdm_sql()` · `app.py`
Apply transforms	Button	Writes cleaned_df and registers table cleaned. Shows first 10 rows.	`page_dwdm_sql()` · `app.py` → `apply_dwdm_transforms()` · `src/dwdm_sql.py`
SQL	Text area	Read-only SELECT / WITH over registered tables (DuckDB, pandas fallback). Example queries are printed above the box.	`page_dwdm_sql()` · `app.py` → `default_sql_examples()` · `src/dwdm_sql.py`
Run SQL	Button	Primary. `run_sql()` → sql_result table. Engine caption shows duckdb or pandas.	`page_dwdm_sql()` · `app.py` → `run_sql()` · `src/dwdm_sql.py`
SQL result	Table	Full result frame after Run SQL.	`page_dwdm_sql()` · `app.py`

17. Button checklist (examiner click-path)

Every `st.button` / `st.download_button` label in app.py, in sidebar-then-page order. If the examiner asks “what does this button do?”, find the page chapter above.

#	Page	Button / download (exact UI label)
1	Sidebar	**Suggest pack from columns**
2	Sidebar	**Save key**
3	Sidebar	**Test Gemini**
4	1. Upload & Clean	**Load Plant sample (default)**
5	1. Upload & Clean	**Load pack demo**
6	1. Upload & Clean	**Apply preset to SQL**
7	1. Upload & Clean	**Load from URL**
8	1. Upload & Clean	**Run industrial clean + 19 quality checks**
9	1. Upload & Clean	**Download Cleaned CSV**
10	2. Joins	**Run join**
11	2. Joins	**Run join chain**
12	3. Map sensors	**Apply mapping**
13	4. Anomaly & RUL	**Detect anomalies + predict RUL**
14	4. Anomaly & RUL	**Optuna tune RUL**
15	4. Anomaly & RUL	**Optuna tune anomalies**
16	5. Charts	**Generate Sensor over time**
17	5. Charts	**Generate Anomaly flags**
18	5. Charts	**Generate Risk by asset**
19	5. Charts	**□ Sensor Correlation Heatmap**
20	5. Charts	**⚠ Anomaly Scatter Plot (warning-emoji + name in UI)**
21	5. Charts	**□ Distribution Histogram**
22	5. Charts	**□ Box Plot by Machine**
23	5. Charts	**□ Rolling Average Trend**
24	5. Charts	**Remove {g_id} (one per saved graph)**
25	6. Insights / Ask	**Refresh brief**
26	6. Insights / Ask	**Gemini polish (wording only)**
27	6. Insights / Ask	**Clear Ask**
28	CAD Twin (APS)	**Load CAD model**
29	CAD Twin (APS)	**Open Anomaly & RUL**
30	CAD Twin (APS)	**Assign selected part to this region**
31	CAD Twin (APS)	**Clear region map**
32	CAD Twin (APS)	**Translate to SVF / get URN**
33	CAD Twin (APS)	**Save URN for this pack**
34	CAD Twin (APS)	**Delete saved URN**
35	Live Connect	**▶ Start live**
36	Live Connect	**□ Stop**
37	Live Connect	**Reset buffer**
38	7. Dashboard	**Export dashboard (HTML) — KPIs + charts + 3D …**
39	Email Report	**Generate Email Preview**
40	Email Report	**Send Email**
41	Email Report	**Download Email as TXT**
42	SQL lab	**Apply transforms**
43	SQL lab	**Run SQL**

3D Twin has no st.button (radio + selectbox + mesh only). Ask page reuses Clear Ask. Chat input on Insights / Ask is not a button — it is `st.chat_input`.

18. Likely viva questions — short honest answers

Q. Is this FADEC or a GCS?

No. FADEC commands the engine; GCS flies the UAV. We mapped SIH26054 as health + twin + fault prediction + mission reliability on sensor rows. Isolation Forest does not move a throttle.

Q. Why 19 checks, not Great Expectations alone?

GE-style expectations are check 12 plus a sub-report. The other 18 cover nulls, constants, zeros, duplicates, z/IQR, a quality-time Isolation Forest, DBSCAN, KMeans, impossible jumps, dead correlations, ydata cardinality, Cleanlab dirty labels, PCA drift, domain physics, association rules, schema, timestamp. Optional libraries (GE / ydata / Cleanlab) can be missing — the proxy still runs.

Q. pandas or Polars?

pandas is the default industrial cleaner. Polars is an opt-in first pass (columnar, no JVM). The 19-stage report always runs on the pandas frame afterwards. PySpark is gone.

Q. Why did Streamlit throw when we set industry_pack after Suggest pack?

The Field selectbox owns key industry_pack. We set _pending_industry_pack and rerun; apply_pending_industry_pack() copies it before the widget is created. Same pattern for pipeline_page (Open Anomaly & RUL) and aps_zip_root_filename / aps_urn_input.

Q. Does CAD Twin redden the whole Rotax when risk is High?

No. Only assigned dbIds or name-matched nodes tint. Solid1-only STEP stays gray. Click a part → Assign selected part to this region. Isolation Forest scores the asset, not the solid.

Q. Will Send Email reach the manager?

Only if SMTP_USER / SMTP_PASSWORD are set and EMAIL_DEMO_MODE is false. Otherwise the file is saved under output/emails/ and the UI says demo mode. That is honest.

Q. What did you write vs what is a library?

We wrote the Streamlit pipeline, industrial cleaner, 19-check report, pack layer, twins, APS glue, and this viva map. Isolation Forest / Random Forest are scikit-learn. Optuna tunes hyperparameters. DuckDB slices URLs. Autodesk APS translates and views CAD. Gemini (optional) polishes wording and Ask. Do not claim you invented Isolation Forest.

End of guide. Source of truth is app.py pages dict + each page_* function. Regenerate: `python docs/generate_viva_guide.py`.